



Developing Benchmarks suite for Jax-Privacy as a Top-level Directory

Author

Neeraj Pathak

neerajrpathak710@gmail.com

Organization

Google DeepMind

0. Table of contents

0. Table of contents

1.About me (summary)

2. Project overview

- Project idea

- Abstract

- Motivation

- Why am I a good fit for the Project?

3. Technical details

- Implementation details

- Project Structure

 - Model selection

 - Dataset Cleansing and structuring

 - Utilities

 - Training Script

 - Testing with edge cases

- CLI Commands and Final Results

- Guides and blogs

4. Deliverables

5. References

6. Other relevant information

- What can my mentors expect from me?

- What are my other commitments?

- About me (extended)

- Use of AI

1.About me (summary)

Name: Neeraj Pathak

Timezone: Indian Standard Time (UTC +5:30)

Email: neerajrpathak710@gmail.com

Country: India

University: Vishwakarma Institute of Technology

Course: B. Tech in Instrumentation & Control Engineering

LinkedIn: <https://www.linkedin.com/in/neeraj-pathak-527a682a9/>

Github: <https://github.com/Neerajpathak07>

Project length: Large (350 hours)

Contact:



2. Project overview

2.1 Project idea

JAX Privacy provides the components to write a variety of mechanisms, but lacks good up-to-date benchmarks that evaluate different mechanisms on standard models / datasets. This project would involve creating a new top-level directory (next to `jax_privacy` and examples). Alongside cleansing of the datasets important to `jax-privacy`, enhancing model definitions and creating a simplistic script to implement DP-SGD.

2.2 Abstract

This project is about creating a simplistic, user-friendly and easily forkable benchmark to test the performance of the `jax-privacy` library and its model on various parameters. The goal of this is to provide the users, even the ones without any technical knowledge with inclusive and easily forkable scripts which can be easy to run across the benchmark suite. So that users can make modifications to test against the baseline.

This benchmark suite will now have the liberty to cleanse various Benchmark datasets which are essential for privacy research. Under this we will be creating different datasets for different units of DP. These will be classified into Example-level, User-level, multi-owner. I decided to opt for these classifications while going through some well-known research papers with citations as listed out in [6.References](#).

Most importantly the project will have immense focus on utilizing in-house Jax components and will take references from the upstream implementations readily present in [jax-privacy](#). Reason being we want the newly created benchmark scripts to align with the current code convention and have proper documentation and unit tests to provide a solid base-line for users in the JAX-Privacy way.

2.3 Motivation

While working on a project in Javascript I discovered that a developer took the challenge of building JAX in Javascript([ref](#)). This really grew my curiosity on what exactly JAX is and what kind of functionalities it provides for its users.

Thankfully due to their open source code and detailed documentation I was able to learn more about this and the various Api's and model's it operates on. This is how I came around Google deepmind and Jax Privacy.

What really motivated me for this project was the need to evaluate my datasets and examples on a higher source of benchmarks when I was contributing to add an example for DP SGD Transformers. Which was something the repository was lacking at the moment.

2.4 Why am I a good fit for the Project?

The reason I believe I would be a good fit for this project is because I bring major expertise in Open Source contributions specifically in Machine Learning and Deep Learning domains. The reason I state this can be proven by my ["100+" Merged PR's](#) in a Scientific and ML computing library called "stdlib".

In the case of having significant experience in fitting in the Jax-privacy environment. In the past few months I have contributed significantly to the repository and created issues and suggested improvements for the codebase. Here is a detailed breakdown:-

-> Issues Created / Resolved

<i>Issue Number</i>	<i>Status</i>
#25	Resolved
#86	Resolved
#124	Resolved

-> Pull-Request Open / Merged

<i>PR ID</i>	<i>Status</i>	<i>Maintainer Comments</i>
#83	Merged	NA
#90	Merged	"I really like what you have done here"
#130	Merged	NA
#131	Merged	NA

-> Provided crucial reviews in my fellow contributors PR's helping them in streamlining their implementations and crossing the finish line.

Ref:-

#107	#126
----------------------	----------------------

-> Additionally my skill set resonates with the project tech-stack which again will be to my advantage in the completion of the project of this stature.

Languages: C++, Java, Python, MySQL

Technologies: React, Node.js, MongoDB, SciPy, Jax.

Concepts: Cloud Computing, CI/CD, Agile, Operating Systems, OOPS, Data Structures and Algorithms.

3. Technical details

These details are based upon the research and exploration I have done regarding the project, as well as the current code convention of the Repository.

During the exploration, I read a few research papers on JAX-privacy models and what are the best known practices for benchmarking on several viable models like, Gemma, LoRA, transformers, etc . Additionally, I took an indepth look at some of the Open source code bases which provided proper documentation on how to write Benchmarks and consolidate a dataset with a given model. For reference links please refer to [6.References](#)

AI was utilized to research and to understand code snippets. Refer to [Use of AI](#) for more information.

3.1 Implementation details

I am planning to implement the Benchmarking suite as a standalone folder at the top-level directory as `benchmarks/` . Under this we will adding 3 major components/folders which will be:-

- `benchmarks/datasets/`:- This will be the core folder to house cleansed benchmarked datasets and different datasets for various units of DP like Example-level, user-level etc.
- `benchmarks/models/`:- This module will contain scripts for various models to test the datasets with. Each model will have an individual script which will be defined using equinox flax nnx Ex:- `cnn.py`, `transformer.py` etc.
- `benchmarks/utis/`:- This will be crucial for Accounting privacy and evaluating metrics for seamless benchmark routines.

For this project, I plan on implementing the benchmark suites as 1 model -> 1 dataset pair. By doing this we will be able to assess each model's performance for a given dataset.

All the implementations suggested in the proposal are structured by taking the Jax code-conventions, core-components and privacy maintenance methodologies into consideration. Throughout the scripts I will be utilizing privacy maintenance components like:- `clipped_grad`, `execution_plan` and `batch_selection` strategy.

By the end I will be providing detailed documentation for the benchmarks folder. This documentation will now help the users on how to use the scripts provided in the suite and by what commands can they be accessed through the CLI. Something like:-

```
bash

$ python train.py --dataset=cifar10 --model=cnn --epsilon=2
```

3.2 Project Structure

This project revolves around developing a benchmark suite which should be easily forked and trained throughout the suite, mainly making it more convenient for researchers. In-order for it to be hassle-free the benchmark folder structure has to be simple and appropriately named, ordered and documented, so that the users would not have a hard-time finding the right file and the purpose of the file.

I plan on keeping the folder structure for `benchmarks/` as:-

```
JAX_PRIVACY/
├── jax_privacy/           # Existing library code
├── examples/             # Existing examples
├── benchmarks/           # NEW: Benchmarking suite
│   ├── datasets/
│   │   ├── __init__.py
│   │   ├── base.py       # Base dataset interface
│   │   ├── example_level.py # CIFAR10, IMDB loaders
│   │   ├── user_level.py  # StackOverflow, FLAIR, CelebA
│   │   └── multi_owner.py # Enron, ArXiv loaders
│   ├── models/
│   │   ├── __init__.py
│   │   ├── base.py       # Base model interface
│   │   ├── cnn.py        # Standard CNN architectures
│   │   ├── transformer.py # Transformer models
│   │   └── lora.py        # LoRA adapter for Gemma
│   ├── train.py          # Main training script
│   ├── config.py         # Configuration classes
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── privacy.py    # Privacy accounting utilities
│   │   └── metrics.py    # Evaluation metrics
└── README.md
```

Model selection

I will be prominently taking 3 models into consideration for this project namely, **CNN, LoRA**

and **transformer**. Will be creating individual scripts for these models and training them w.r.t the datasets and different DP units.

- **cnn.py**:- Implementing a simple 3-4 conv layers model which will be utilizing a neural net library such as flax nnx. Again this will be easily configurable through hyperparameters.
- **transformer.py**:- This model will have Configurable depth, heads, embedding dimension and provide support for both classification and sequence tasks. Will be utilizing the following Jax components:- Multi-head attention, Feed-forward layers, Positional encoding, Layer normalization.
- **lora.py**:- Since LoRA acts as an adapter for Gemma I will be leveraging an open-source weight pretrained model and adding low-rank adapter layers. Only training adapter parameters as this is privacy-efficient.

Alongside every model definition I will also be defining a loss function whose signature is consistent across both models, e.g.,

```
def loss_fn(params, batch, prng, *, graphdef):
    """
    Args:
        params: Model parameters for Flax structure
        batch: Dict with 'inputs' and 'labels' keys
        prng: JAX random key for dropout/augmentation
        graphdef: Model architecture definition

    Returns:
        Scalar loss value
    """
    logits = model.apply(graphdef, params, batch['inputs'], prng)
    loss = cross_entropy(logits, batch['labels'])
    return loss
```

Since here we are using multiple models for training alongside multiple-datasets. Handling these inside the training script would be chaotic and unreasonable. To resolve this, I plan on creating a base file namely:- **models/base.py** . This script is intended to pool in all the model classes and Standardized Loss Function Signature. This is also the MOST CRITICAL reason, DP-SGD requires computing per-example gradients.

To have a clear idea on how it will be implemented ref:-


```

from typing import Callable, Any
import jax
import jax.numpy as jnp

# Standard loss function signature
LossFn = Callable[[Any, Dict[str, jnp.ndarray], jax.random.PRNGKey, Any],
jnp.ndarray]

class BenchmarkModel:
    """Base interface for benchmark models."""

    def __init__(self, num_classes: int, **kwargs):
        self.num_classes = num_classes

    def init(self, prng_key: jax.random.PRNGKey, input_shape: Tuple[int,
...]):
        """Initialize model parameters."""
        raise NotImplementedError

    def get_loss_fn(self) -> LossFn:
        """Returns loss function with standard signature."""
        raise NotImplementedError

```

Dataset cleansing and structuring

Based on my previous experiments, I anticipate that handling data will be one of the most challenging parts of this project. This difficulty arises primarily because datasets come in numerous entities . Since the project's primary goal is to simplify benchmarking, providing users with a wide variety of Benchmarking datasets is a crucial step.

To utilize a wide variety of datasets, creating individual scripts for these would create chaos around the project structure and make it difficult for users to iterate through every dataset. To ease this I will be segregating these according to different units of DP as:-

- `datasets/example_level.py`:- For the example level datasets will be opting for the ones which are very important for JAX Privacy. These would be:- CIFAR10 & IMDB.
 - **Why only these datasets?**:- CIFAR-10 and IMDB are included as canonical example-level DP benchmarks. CIFAR-10 is the de-facto standard for evaluating DP-SGD, while IMDB provides a lightweight NLP task for evaluating sequence models under privacy constraints. Together, they

provide fast, reproducible, and literature-comparable benchmarks across vision and text modalities.

- CIFAR10 and IMDB can be implemented in this script as follows:-

```
def download_and_clean_cifar10(cache_dir: str) -> Tuple[np.ndarray,
np.ndarray]:
    # Download, normalize to [0,1], convert to JAX arrays

def download_and_clean_imdb(cache_dir: str, vocab_size: int = 10000):
    # Download, tokenize, pad sequences, create vocab
```

This type of definition will be consistently used across the dataset scripts. Reason being while asserting the benchmarks at the top-level. We can now directly download and clean the dataset through a single definition source without any extra-hassle. Making this more efficient and easy-to use.

- `datasets/user_level.py`:- This dataset level will group examples by user ID, handle a variable number of examples per user and support user-level sampling for DP-SGD. Prominent examples of benchmarking datasets under this level would be:- StackOverflow, Enron where owner = sender, FLAIR and CelebA.
- `datasets/multi_owner.py`:- At this level the datasets are used to track multiple owners per example. By doing this we can now provide support for differential privacy across all owners. Standard multi-owner datasets like Enron where owner = all participants and Arxiv abstracts where owner = all authors will be utilized in this script.

Since we are dealing with vast varieties of datasets and types you might ask: How are we able to manage these at once across the entire suite? Well the solution to this is creating a `datasets/base.py` script. The purpose of which is-imagine if you are creating a benchmarking suite for multiple-dataset but in which each data class was implemented differently. Now the top-level training script will be a mess of just if statements.

Under a singular base file we get the opportunity to group together all the different data classes. So inside the training script instead of calling every dataset and test against it. We can simply call the base class and use this as a parameter for training. This also makes the training script much more readable and easy to maintain. Similarly as it was done for [Model Selection](#).

To get an idea of how it will be implemented Ref:-

```
from typing import Iterator, Tuple, Dict, Any
import jax.numpy as jnp
from dataclasses import dataclass
```

```

@dataclass
class PrivacyUnit:
    """Defines the granularity of privacy protection."""
    level: str # 'example', 'user', 'multi-owner'
    id_field: str # Field name containing the privacy unit ID

class BenchmarkDataset:
    """Base interface for all benchmark datasets."""

    @property
    def privacy_unit(self) -> PrivacyUnit:
        raise NotImplementedError

    def get_train_iter(self, batch_size: int) -> Iterator[Dict[str,
jnp.ndarray]]:
        raise NotImplementedError

    def get_test_iter(self, batch_size: int) -> Iterator[Dict[str,
jnp.ndarray]]:
        raise NotImplementedError

    @property
    def num_classes(self) -> int:
        raise NotImplementedError

```

Utilities

The utilities present inside `utilities/` are intended to act as the backbone of the infrastructure which prevents code duplication and provides shared functionality across all datasets, models, and training scripts. Without it, every component would need to reimplement common operations like privacy accounting, gradient clipping, configuration management, and metrics computation, leading to inconsistent implementations, bugs, and maintenance nightmares.

I am majorly planning on adding 2 crucial utils namely:-

- `utilities/privacy.py`:- The privacy engine that computes noise multipliers from (ϵ, δ) budgets, tracks cumulative privacy expenditure using SGD accounting, calculates privacy amplification from subsampling, and validates privacy parameters. This is critical because incorrect privacy accounting means either violating privacy guarantees (underestimating ϵ) or getting terrible utility (overestimating ϵ).

Centralizing this logic ensures all training runs use correct, consistent privacy math.

- `utilities/metrics.py`:- Provides standardized evaluation metrics (accuracy, loss, F1, confusion matrices) that works across all datasets and tasks, formats results for logging and visualization, computes privacy-specific metrics like membership inference vulnerability, and generates comparison tables for benchmarking. This ensures fair comparisons across different experiments by using identical metric computation. Here my primary focus will be on developing Timing metrics which would process - `per_step_time`, `compilation_time` & total FLOPs (using `jax cost_analysis`).

Training Script

This training script is the essence of this project. So far every model and datasets which we have selected and created at the top-level directory inside `benchmarks/` will now be tested and trained through running this script from the base-line. This script is meant to be easily forkable and can be run easily across the entire bench suite. The intention behind this is to provide users and researchers with the convenience of testing each model with each dataset.

An **important point** that I want to state is that I will be programming this against the `execution_plan` abstraction. Reason being there will be a standard way to swap out different implementations present in `jax_privacy/experimental/execution_plan.py`. As of now we only have support for BandMF but are expecting to add a few more in the future.

So far considering the use case of 3 models and 8 datasets users now have the freedom of evaluating, training and experimenting each dataset with a standard model. Eg:- training a transformer on cifar10. This can again easily be run through CLI using a single command line.

To have a clear idea on what the training logic looks like at the core Refer to the given code block:-

- **Training loop interface:-**

```
@jax.jit
def dp_train_step(
    state: TrainState,
    batch_data: Tuple[jax.Array, jax.Array],
    is_padding_example: jax.Array
) -> Tuple[TrainState, jax.Array]:
    batch_x, batch_y = batch_data
    grads, aux = grad_fn(
        state.params,
        batch_x,
```



```

parser.add_argument('--clipping_norm', type=float, default=1.0,
                    help='Per-example gradient clipping threshold')
parser.add_argument('--noise_multiplier', type=float, default=None,
                    help='Noise multiplier (computed from epsilon if not set)')

# Training hyperparameters
parser.add_argument('--batch_size', type=int, default=256)
parser.add_argument('--epochs', type=int, default=50)
parser.add_argument('--learning_rate', type=float, default=0.01)
parser.add_argument('--weight_decay', type=float, default=0.0)

parser.add_argument('--seed', type=int, default=42)
parser.add_argument('--output_dir', type=str, default='./results')
parser.add_argument('--checkpoint_freq', type=int, default=5)

return parser

```

Testing with edge cases.

Since we want the upcoming suite to be reused and work seamlessly for the DP piping. The best practice to maintain and keep the code intact would be to test these against a few edge cases.

How do I plan on doing that?

I will be creating a benchmarks folder inside the upstream tests folder which will look like:- `tests/benchmarks/` by doing this we can explicitly have a dedicated section to hold all our test files. I plan on adding test cases primarily for the following modules:-

- `test_datasets.py`:- Here we will be able to check whether or not the dataset was loaded properly or it commutes the correct `batch_shape`, `vocabs`, etc. To get an idea on how this can be tested refer:-

```

def test_load_cifar10_shapes():
    x, y = load_cifar10()
    assert x.shape[0] == y.shape[0]

```

- `test_models.py`:- This test file plays a crucial role in testing the various models that are very important to JAX Privacy. These tests will ensure that forward pass runs smoothly, output shape is correct and JIT works correctly. Ref:-

```
def test_transformer_forward():
    params = init_model_params(key)
    x = jnp.ones((2, 128), dtype=jnp.int32)
    logits = model(params, x)
    assert logits.shape == (2, 128, vocab_size)
```

- `test_loss.py`:- Here one might ask **why we are testing loss_fcn at all?**. The reason being is that our entire training and privacy accounting depends on the loss function being correct which makes this very viable. If in turn `loss_fcn` is correct then automatically other essential privacy components like:- gradients, clippers and noise would also be correct. Ref:-

```
def test_loss_scalar():
    loss = loss_fn(params, batch_x, batch_y)
    assert loss.ndim == 0
```

- `test_training.py`:- Lastly the most crucial step is validating if our training script runs clean or not. As if it does not then we won't be able to utilize any benchmarks nor can we provide CLI commands for users for training. Hence, this file plays an important role in governing the state of the project. Ref:-

```
def test_train_step_updates_params():
    new_params, *_ = dp_train_step(...)
    assert not jnp.allclose(new_params["w"], params["w"])
```

Note:- The test snippets suggested here are drafted by referring to the upstream test implementations present in jax-privacy.

Reference points-

- https://github.com/google-deepmind/jax_privacy/blob/main/tests/batch_selection_test.py
- https://github.com/google-deepmind/jax_privacy/blob/main/tests/gradient_clipping_test.py

CLI Commands and Final Results

Now that the essential components for benchmarking are all in-place and given that these are provided security under JAX-privacy methodologies. We can now configure the Command-line which is meant to be used in the CLI for getting desired output.

Sample CLI-command would be:-

```
bash

$ python train.py --dataset=cifar10 --model=cnn --epsilon=2
$ python train.py --dataset=imdb --model=lora_gemma --epsilon=2
$ python train.py --dataset=cifar10 --model=transformer --epsilon=2
.
.
```

For advanced research purposes researchers can use command lines like:-

```
bash

$ python train.py --dataset stackoverflow --model transformer --epsilon 5.0
--epochs 40

$ python train.py --dataset arxiv --model lora_gemma --epsilon 8.0 --epochs
25

$ python train.py --dataset cifar10 --model cnn --epsilon 3.0 --delta 1e-5
--batch_size 256 --epochs 50 --learning_rate 0.01
```

But again a question might arise: how would the new users or researchers get to know about these CLI commands and how to use them?. For which I intended to create a detailed Documentation along with Guides to provide proper direction to user-queries refer [3.3 Guides and Blogs](#) for more information.

Now that we know how various CLI commands can be utilized. Let's take a look at a sample use case of the Benchmarking suite.

```
python train.py --dataset=cifar10 --model=cnn --epsilon=3.0 --epochs=50
\\
```

```
### Console Output:
\\
```

```
=====
JAX Privacy DP-SGD Benchmarking Suite
=====
```

```
Configuration:
```

```
Dataset: cifar10
Model: cnn
Privacy:  $\epsilon=3.0$ ,  $\delta=1e-05$ 
Batch Size: 256
```



```

Epochs: 50
Learning Rate: 0.01

Loading dataset: cifar10...
  Training examples: 50,000
  Test examples: 10,000
  Privacy unit: example-level
  Input shape: (32, 32, 3)
  Number of classes: 10

Initializing model: cnn...
  Architecture: [32, 64, 128] channels
  Parameters: 1,234,567
  Dropout rate: 0.1

Computing noise multiplier for target privacy...
  Target  $\epsilon$ : 3.0
  Target  $\delta$ : 1e-05
  Total steps: 9,765 (50 epochs  $\times$  195 batches)
  Sample rate: 0.00512
   $\rightarrow$  Noise multiplier: 1.247

Initializing training for cifar10 with cnn...
Starting training for 50 epochs...

Epoch 1/50 | Train Loss: 2.1234 | Train Acc: 0.2145 | Test Acc: 0.2301 |  $\epsilon$ : 0.14 | Time: 12.34s
Epoch 5/50 | Train Loss: 1.7856 | Train Acc: 0.3567 | Test Acc: 0.3712 |  $\epsilon$ : 0.68 | Time: 11.89s
Epoch 10/50 | Train Loss: 1.5234 | Train Acc: 0.4523 | Test Acc: 0.4678 |  $\epsilon$ : 1.32 | Time: 11.67s
Epoch 15/50 | Train Loss: 1.3456 | Train Acc: 0.5234 | Test Acc: 0.5389 |  $\epsilon$ : 1.89 | Time: 11.54s
Epoch 20/50 | Train Loss: 1.2123 | Train Acc: 0.5789 | Test Acc: 0.5912 |  $\epsilon$ : 2.41 | Time: 11.48s
Epoch 25/50 | Train Loss: 1.1234 | Train Acc: 0.6123 | Test Acc: 0.6234 |  $\epsilon$ : 2.87 | Time: 11.52s
Epoch 30/50 | Train Loss: 1.0567 | Train Acc: 0.6456 | Test Acc: 0.6501 |  $\epsilon$ : 3.28 | Time: 11.46s
Privacy budget exhausted at epoch 30. Stopping.

Training complete! Running final evaluation...

Final Test Accuracy: 0.6501
Final Privacy:  $\epsilon$ =3.28,  $\delta$ =1.00e-05

Total training time: 6 minutes 47 seconds
=====

```

Note:- This is an over the top understanding of how the Console Output will look like as per the given CLI interface logic provided in `train.py`. But I am open to suggestions on refactoring these as the mentors suggest.

3.3 Guides and Blogs

Even with a simple and user-friendly benchmarking suite, experimenting can still feel overwhelming, especially for users who are new to it. To make the process even more approachable, I plan to write a series of beginner-friendly guides and blogs that explain the concepts behind each feature. These won't dive deep into complex math or heavy technical details but will instead focus on delivering just enough understanding to help users make informed decisions.

These guides will be comprehensive yet easy to follow, aimed at answering common questions like:

"What all models can I select?"

"What type of dataset can it be trained against?"

"Which hyperparameters can be useful in evaluating models for accuracy?"

"What is LoRA, and why should I use it?"

I'll be using [readthedocs](#) to write and publish these guides. This is a new tool for me as well and I'm excited to learn something new through this project as well. The reason why I am opting for this is due to their interesting feature in which we can import our project effortlessly and create a Yaml file which will build the docs after every single commit. This reduces the extra work for maintaining the docs after every major release.

Here are a few Blog topics I plan to write during the project:

- Choosing the right base-line model for your benchmarking.
- Understanding the different units of DP benchmark datasets.
- Understanding the necessity of benchmarking and evaluating different models.
- LoRA 101

4. Deliverables

Standalone Benchmarking Suite:

A fully functional benchmarking suite to be placed at the top-level directory alongside `jax_privacy/` & `examples/`. Providing 3 different models and 8 different varieties of datasets to select and experiment on. Most importantly implementing this with respect to Jax-privacy methodologies and core components.

Guides and Blogs:

A collection of beginner-friendly guides and blogs to help users understand key concepts around benchmarking. These will make it easier for users to make calculated evaluations without any need of prior or technical knowledge.

5. References

- <https://research.google/blog/differentially-private-machine-learning-at-scale-with-jax-privacy/>
- https://www.researchgate.net/publication/385766245_ReLax_Efficient_and_Scalable_Recourse_Explanation_Benchmarking_using_JAX
- <https://arxiv.org/abs/2211.04110>
- <https://openreview.net/pdf?id=hw2VfWAr6t6>
- <https://cs.uwaterloo.ca/~s3mohapa/papers/DPBenchmarks.pdf>

Open Source Repositories:-

- <https://github.com/TheSalon/fast-dpsgd>
- <https://github.com/DmsKinson/DPMLBench>
- <https://github.com/sunblaze-ucb/dpml-benchmark>

6. Other Relevant Information

What can my mentors expect from me?

- Complete transparency of the progress and weekly updates through well organized progress reports.
- Delivering accurate & quality work before the proposed deadline.
- Taking guidance from the mentors form open-ears and working on the suggested changes on priority.

What are my other commitments?

As of now I do not have any extended commitments or employment expectations anywhere. Hence, I can fully devote my summer to this project and the organization.

About Me (extended)

I am a 3rd year Undergraduate student at Vishwakarma Institute of Technology, currently pursuing B.Tech in Instrumentation & Control Engineering. My initial interest in programming developed in my Sophomore year when I started learning C/C++ & Python and also learning core concepts like data structures and algorithms, Object-oriented programming and Database Management systems.

By the end of my Sophomore year I entered the universe of Web Development acquiring firm knowledge and practice of Javascript, Typescript, React, Next, Vue, Redis, MongoDB and Express.js. Which helped me extensively in the early days of my 2nd year to build projects and work on newer technologies.

Starting to contribute to open-source was also one of the best decisions to help me get good

hands-on practice for coding and learning along the way. I am also keen on learning cutting edge technologies like DevOps, Machine Learning, Artificial Intelligence and Data Science.

Use of AI

I have used AI in prominently 2 things first being the research part such as finding out all the relevant research papers for this project. Then a little assistance on learning about the different models and how they operate in general. Lastly for understanding code snippets from upstream source codes these codes are prominently based on what are the best practices for benchmarking and also how Jax-privacy maintains its nature across platforms.